# Numerical techniques in Hydrodynamics.

## Tutorial - 6

## Lid - Driven cavity problem

By:
Karthick. J
OE23S018
M.S. Scholar.

In compressible fluid flow :-

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \rightarrow \text{conservation of mass.}$$

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial x} + \nu/\rho\left[\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right]$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial y} + v\frac{\partial v}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial x} + \nu/\rho\left[\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right]$$

↳ momentum Equations For x and Y direction.

## Vorticity - Stream Function

* Vorticity is measure the amount of Rotation in flow

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial v}$$

the fluid Rotation about the z axis.

$$\omega_z = \left[\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}\right]$$

u, v values substitude in the continuity Equation,

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0.$$

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

$$\frac{\partial \left(\frac{\partial \psi}{\partial y}\right)}{\partial x} + \frac{\partial \left(-\partial \psi / \partial x\right)}{\partial y} = 0$$

$\omega \rightarrow$ curl of velocity.

$$\frac{\partial \psi}{\partial x \partial y} - \frac{\partial \psi}{\partial x \partial y} = 0$$

flow is conserved property.

the continuity is equal to zero.

$$\nabla \cdot \vec{V} = 0$$

For the vorticity ($\omega$)

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}.$$

$$\omega = -\frac{\partial \left(\partial \psi / \partial x\right)}{\partial x} - \frac{\partial \left(\frac{\partial \psi}{\partial y}\right)}{\partial y}$$

$$\omega = \frac{\partial^2 \psi}{\partial x^2} - \frac{\partial^2 \psi}{\partial y^2}$$

$$\boxed{\omega = -\left[\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2}\right]}$$

$$\boxed{\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = -\omega} \rightarrow \text{poisson's equation for stream function.}$$

For Eliminating the pressure term,

x momentum equation $\frac{\partial}{\partial y}$ and y momentum equation $\partial/\partial x$

$$\frac{\partial}{\partial t}\left(\frac{\partial u}{\partial y}\right) + \frac{\partial}{\partial y}(\vec{V}\nabla\vec{u}) = -\frac{1}{\rho}\frac{\partial p}{\partial x}\left(\frac{\partial}{\partial y}\right) + \frac{\nu}{\rho}\left(\frac{\partial}{\partial y}\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right)\right)$$

$$(-)$$

$$\frac{\partial}{\partial t}\left(\frac{\partial v}{\partial x}\right) + \frac{\partial}{\partial x}(\vec{V}\nabla v) = -\frac{1}{\rho}\frac{\partial p}{\partial y}\left(\frac{\partial}{\partial x}\right) + \frac{\nu}{\rho}\left(\frac{\partial}{\partial x}\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right)\right)$$

---

$$\frac{\partial}{\partial t}\left(\frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}\right) + \nabla\cdot\nabla(\omega) = \frac{1}{Re}\left(\nabla^2\omega\right)$$

$$\frac{\partial \omega}{\partial t} + u\left(\frac{\partial \omega}{\partial x}\right) + v\frac{\partial \omega}{\partial y} = \frac{1}{Re}\left(\frac{\partial^2\omega}{\partial x^2} + \frac{\partial^2\omega}{\partial y^2}\right)$$

$$\frac{\partial \omega}{\partial t} + \left[\frac{\partial \varphi}{\partial y}\left(\frac{\partial \omega}{\partial x}\right) + \frac{\partial \varphi}{\partial x}\frac{\partial \omega}{\partial y}\right] = \frac{1}{Re}\left(\frac{\partial^2\omega}{\partial x^2} + \frac{\partial^2\omega}{\partial y^2}\right)$$

$$\frac{\partial^2\varphi}{\partial x^2} + \frac{\partial^2\varphi}{\partial y^2} = -\omega$$

$$\left[\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2}\right] = \rho\left[\left(\frac{\partial^2\varphi}{\partial x^2}\right)\left(\frac{\partial^2\varphi}{\partial y^2}\right) - \left(\frac{\partial^2\varphi}{\partial x\partial y}\right)^2\right]$$

$$Re = \frac{UL}{\nu}$$

# Discretize equations

## Stream Function Formula.

$$u = \frac{\partial \psi}{\partial y} \Rightarrow \frac{\psi_{i,j+1} - \psi_{i,j-1}}{2\Delta y}$$

$$v = \frac{\partial \psi}{\partial x} \Rightarrow \frac{\psi_{i+1,j} - \psi_{i,j}}{2\Delta x}$$

## Vorticity Transport equation

$$\frac{\partial \omega}{\partial t} + \left[ u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} \right] = \frac{1}{Re} \left[ \frac{\partial^2 \omega}{\partial x^2} + \frac{\partial^2 \omega}{\partial y^2} \right]$$

① Convection term :- (central difference)

$$\left[ u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} \right] = \left[ \frac{u^n_{i+1,j} \omega^n_{i+1,j} - u^n_{i-1} \omega^n_{i-1,j}}{2\Delta x} \right] + \left[ \frac{v^n_{j+1} \omega^n_{j+1} - v_{j+1} \omega^n_{j+1}}{2\Delta y} \right]$$

② Diffusion term. (central difference)

$$\frac{1}{Re} \left[ \frac{\partial^2 \omega}{\partial x^2} + \frac{\partial^2 \omega}{\partial y^2} \right] \Rightarrow \frac{1}{Re} \left[ \frac{\omega^n_{i+1,j} - 2\omega^n_{ij} + \omega^n_{i-1,j}}{(\Delta x)^2} + \frac{\omega^n_{ij+1} - 2\omega^n_{ij} + \omega^n_{ij-1}}{(\Delta y)^2} \right]$$

For the temporal solution, Explicit Scheme.

$$\frac{w_{ij}^{n+1} - w_{ij}^{n}}{\Delta t} \Rightarrow \frac{\partial w_{ij}}{\partial t}$$

Substituting the Transport equation,

$$\frac{w_{ij}^{n+1} - w_{ij}^{n}}{\Delta t} + \frac{u_{i+1j}^{n} w_{i+1j}^{n} - u_{i-1j}^{n} w_{i-1j}^{n}}{2\Delta x} + \frac{v_{ij+1}^{n} w_{ij+1}^{n} - v_{ij-1} w_{ij-1}}{2\Delta y}$$

$$= \frac{1}{Re}\left[\left[\frac{w_{i+1j}^{n} - 2w_{ij}^{n} + w_{i-1j}^{n}}{\Delta x^2}\right] + \left(\frac{w_{ij+1}^{n} - 2w_{ij} + w_{ij-1}}{\Delta y^2}\right)\right]$$

$$w_{ij}^{n+1} = w_{ij}^{n} + \frac{\Delta t}{Re}\left[\frac{w_{i+1j}^{n} - 2w_{ij}^{n} + w_{i-1j}^{n}}{\Delta x^2} + \frac{w_{ij+1} - 2w_{ij} + w_{ij-1}}{\Delta y^2}\right]$$

$$- \frac{\Delta t}{2\Delta x}\left[u_{i+1j}^{n} w_{i+1j}^{n} - u_{i-1j}^{n} w_{i-1j}^{n}\right] - \Delta t\left(\frac{v_{ij+1}^{n} w_{ij+1}^{n} - v_{ij-1}^{n} w_{ij-1}^{n}}{2\Delta y}\right)$$

For the Poisson's equation

$$\frac{\psi_{i+1}^{n} - 2\psi_{ij}^{n} + \psi_{i-1j}^{n}}{(\Delta x)^2} + \frac{\psi_{ij-1}^{n} - 2\psi_{ij}^{n} + \psi_{ij-1}^{n}}{\Delta y^2} = -w_{ij}^{n}.$$

For von-neumann. stability analysis of Transport equation. [Refered From, Lid driven cavity flow using stencil based Numerical method – Hannes Juujärvi, Isak Kinnunen.

$$\Delta t \leq \frac{h^2 Re}{4}$$

$h = \Delta x = \Delta y$. for uniform grid.

****Algorithm for Lid-Driven Cavity Flow Solver (Vorticity-Stream Function Approach) ****

Algorithm: Lid-Driven Cavity Flow Solver (Vorticity-Stream Function Method)

1. Initialize Simulation Parameters
   - Domain size L, grid spacing dx/dy
   - Reynolds number Re, viscosity μ, density ρ
   - Time step dt, max iterations, convergence threshold
   - Lid velocity U_wall

2. Allocate Memory
   - Vorticity array ζ[Nx][Ny]
   - Stream function Ψ[Nx][Ny]
   - Velocity components u[Nx][Ny], v[Nx][Ny]
   - Previous vorticity ζ_prev[Nx][Ny]

3. Initialize Arrays to Zero
   ζ ← 0, Ψ ← 0, u ← 0, v ← 0

4. Main Simulation Loop
   for iter = 1 to MaxIter do
      4.1 Apply Boundary Conditions (Equations 30-33 from PDF)
         - Top wall: ζ_top = -2Ψ_second_last_row/dy² - 2U_wall/dy
         - Bottom: ζ_bottom = -2Ψ_second_row/dy²
         - Left: ζ_left = -2Ψ_second_col/dx²
         - Right: ζ_right = -2Ψ_second_last_col/dx²

4.2 Store Previous Vorticity

$\zeta_prev \leftarrow \zeta$

4.3 Update Vorticity (Equation 16 from PDF)

for all interior points (i,j) do

$\partial\Psi/\partial x = (\Psi[i+1,j] - \Psi[i-1,j])/(2dx)$

$\partial\Psi/\partial y = (\Psi[i,j+1] - \Psi[i,j-1])/(2dy)$

$\nabla^2\zeta = (\zeta_prev[i+1,j] + \zeta_prev[i-1,j]$

$+ \zeta_prev[i,j+1] + \zeta_prev[i,j-1]$

$- 4\zeta_prev[i,j])/(dx^2)$

$\zeta[i,j] = \zeta_prev[i,j] + dt * [$

$(\partial\Psi/\partial y * (\zeta_prev[i+1,j] - \zeta_prev[i-1,j])/(2dx))$

$- (\partial\Psi/\partial x * (\zeta_prev[i,j+1] - \zeta_prev[i,j-1])/(2dy))$

$+ (\mu/\rho) * \nabla^2\zeta$

]

end

4.4 Solve Stream Function (Equation 18 from PDF)

Red-Black Gauss-Seidel Iteration:

for phase = 0 to 1 do

for all points (i,j) where (i+j)%2 == phase do

$\Psi[i,j] = 0.25 * [$

$\Psi[i+1,j] + \Psi[i-1,j]$

$+ \Psi[i,j+1] + \Psi[i,j-1]$

$+ dx^2 * \zeta[i,j]$

]

end

end

4.5 Check Convergence

if iter > 10 then

max_error = max(|ζ - ζ_prev|)

if max_error < tolerance then

break

end

end

4.6 Periodic Output

if iter % output_interval == 0 then

Calculate velocities (Equations 34-35):

u = ∂Ψ/∂y, v = -∂Ψ/∂x

Write Tecplot file with (x,y,u,v,ζ,Ψ)

end

end

5. Final Output

Calculate velocities

Write final Tecplot file

6. Visualize Results

- ParaView: Load Tecplot files, create contours/streamlines

- Plot velocity profiles and vorticity distribution

## Lid-driven Cavity Code:

```
#include <stdio.h>
#include <math.h>
#include <cuda.h>
#include <cuda_runtime.h>
#include <sys/stat.h>

// Simulation Parameters
#define L 1.0
#define DX 0.0025
#define DY 0.0025
#define Re 100.0
#define Dt 0.001
#define MaxIter 50000
#define Error 1e-5
#define U_Wall 1.0
#define NX ((int)(L/DX) + 1)
#define NY ((int)(L/DY) + 1)
#define OUTPUT_INTERVAL 1

// CUDA Error Checking
#define CHECK(call) { \
    cudaError_t err = call; \
    if(err != cudaSuccess) { \
        fprintf(stderr, "CUDA Error: %s:%d - %s\n", __FILE__, __LINE__, 
cudaGetErrorString(err)); \
```

```
        exit(1); \
    } \
}

// Kernel Prototypes with C linkage
#ifdef __cplusplus
extern "C" {
#endif
__global__ void apply_boundary_conditions(double *Vort, double *Str);
__global__ void update_vorticity(double *Vort, double *Vop, double *Str);
__global__ void solve_stream_function(double *Str, double *Vort, int phase);
__global__ void calculate_velocity(double *u, double *v, double *Str);
#ifdef __cplusplus
}
#endif

// ParaView-compatible Tecplot output
void write_tecplot2D(double* u, double* v, double* vort, double* stream,
                     int nx, int ny, double dx, double dy, int iter) {
    char filename[256];
    FILE *fp;

    // Create output directory
    struct stat st = {0};
    if (stat("output", &st) == -1) mkdir("output", 0700);

    sprintf(filename, "output/flow_%04d.dat", iter);
```

```
    fp = fopen(filename, "w");

    // Write header
    fprintf(fp, "TITLE = \"Lid-Driven Cavity Flow\"\n");
    fprintf(fp, "VARIABLES = \"X\", \"Y\", \"U\", \"V\", \"Vorticity\",
\"Stream\"\n");
    fprintf(fp, "ZONE I=%d, J=%d, DATAPACKING=POINT\n", nx, ny);

    // Write data with fixed-width format
    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) {
            double x = i * dx;
            double y = j * dy;
            int idx = j * nx + i;
            fprintf(fp, "%10.6e %10.6e %10.6e %10.6e %10.6e %10.6e\n",
                    x, y,
                    u[idx], v[idx],
                    vort[idx], stream[idx]);
        }
    }
    fclose(fp);
}

int main() {
    // Allocate Unified Memory
    double *Vort, *Vop, *Str, *u, *v;
    CHECK(cudaMallocManaged(&Vort, NX*NY*sizeof(double)));
    CHECK(cudaMallocManaged(&Vop, NX*NY*sizeof(double)));
```

```
    CHECK(cudaMallocManaged(&Str, NX*NY*sizeof(double)));
    CHECK(cudaMallocManaged(&u, NX*NY*sizeof(double)));
    CHECK(cudaMallocManaged(&v, NX*NY*sizeof(double)));

    // Initialize Arrays
    for(int i=0; i<NX*NY; i++) {
        Vort[i] = 0.0; Vop[i] = 0.0; Str[i] = 0.0; u[i] = 0.0; v[i] = 0.0;
    }

    // CUDA Grid Configuration
    dim3 block(16, 16);
    dim3 grid((NX + block.x - 1)/block.x, (NY + block.y - 1)/block.y);

    // Main Simulation Loop
    int converged = 0;
    for(int iter=0; iter<MaxIter && !converged; iter++) {
        // Apply Boundary Conditions first
        apply_boundary_conditions<<<grid, block>>>(Vort, Str);
        CHECK(cudaDeviceSynchronize());

        // Store previous vorticity (with BCs applied)
        CHECK(cudaMemcpy(Vop, Vort, NX*NY*sizeof(double),
cudaMemcpyDeviceToDevice));

        // Update Vorticity using previous state (Vop)
        update_vorticity<<<grid, block>>>(Vort, Vop, Str);
        CHECK(cudaDeviceSynchronize());
```

```
    // Solve Stream Function (Red-Black Gauss-Seidel)
    for(int phase=0; phase<2; phase++) {
        solve_stream_function<<<grid, block>>>(Str, Vort, phase);
        CHECK(cudaDeviceSynchronize());
    }

    // Check convergence every iteration after 10 steps
    if(iter > 1) {
        double max_error = 0.0;
        for(int idx=0; idx<NX*NY; idx++) {
            double error = fabs(Vort[idx] - Vop[idx]);
            if(error > max_error) max_error = error;
        }
        printf("Iter %d: Max Error = %e\n", iter, max_error);
        if(max_error < Error) {
            converged = 1;
            printf("Converged after %d iterations\n", iter);
        }
    }

    // Output every OUTPUT_INTERVAL steps
    if(iter % OUTPUT_INTERVAL == 0) {
        calculate_velocity<<<grid, block>>>(u, v, Str);
        CHECK(cudaDeviceSynchronize());
        write_tecplot2D(u, v, Vort, Str, NX, NY, DX, DY, iter);
    }
}
```

```
    // Final Output
    calculate_velocity<<<grid, block>>>(u, v, Str);
    CHECK(cudaDeviceSynchronize());
    write_tecplot2D(u, v, Vort, Str, NX, NY, DX, DY, MaxIter);

    // Free Memory
    CHECK(cudaFree(Vort));
    CHECK(cudaFree(Vop));
    CHECK(cudaFree(Str));
    CHECK(cudaFree(u));
    CHECK(cudaFree(v));

    return 0;
}

// Boundary Conditions Kernel (equations 30-33 from PDF)
__global__ void apply_boundary_conditions(double *Vort, double *Str) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if(i >= NX || j >= NY) return;

    const double h2 = DX * DX;

    // Top Boundary (equation 30)
    if(j == NY-1 && i > 0 && i < NX-1) {
```

```
        Vort[i*NY + j] = (-2*Str[i*NY + (j-1)]/h2) - (2*U_Wall/DX);
    }
    // Bottom Boundary (equation 31)
    else if(j == 0 && i > 0 && i < NX-1) {
        Vort[i*NY + j] = -2*Str[i*NY + (j+1)]/h2;
    }
    // Left Boundary (equation 32)
    else if(i == 0 && j > 0 && j < NY-1) {
        Vort[i*NY + j] = -2*Str[(i+1)*NY + j]/h2;
    }
    // Right Boundary (equation 33)
    else if(i == NX-1 && j > 0 && j < NY-1) {
        Vort[i*NY + j] = -2*Str[(i-1)*NY + j]/h2;
    }
}

// Vorticity Update Kernel (equation 16 from PDF)
__global__ void update_vorticity(double *Vort, double *Vop, double *Str) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if(i <= 0 || i >= NX-1 || j <= 0 || j >= NY-1) return;

    const double inv2h = 1.0/(2.0*DX);
    const double h2 = DX*DX;

    // Calculate derivatives using previous state (Vop)
```

```
    double dStr_dy = (Str[i*NY + (j+1)] - Str[i*NY + (j-1)]) * inv2h;
    double dStr_dx = (Str[(i+1)*NY + j] - Str[(i-1)*NY + j]) * inv2h;
    double dVop_dx = (Vop[(i+1)*NY + j] - Vop[(i-1)*NY + j]) * inv2h;
    double dVop_dy = (Vop[i*NY + (j+1)] - Vop[i*NY + (j-1)]) * inv2h;

    // Calculate Laplacian (equation 16)
    double laplacian = (Vop[(i+1)*NY + j] + Vop[(i-1)*NY + j] +
                        Vop[i*NY + (j+1)] + Vop[i*NY + (j-1)] -
                        4*Vop[i*NY + j]) / h2;

    // Update vorticity (equation 16)
    Vort[i*NY + j] = Vop[i*NY + j] + Dt * (
        (laplacian/Re) - (dStr_dy*dVop_dx - dStr_dx*dVop_dy)
    );
}

// Stream Function Solver (equation 18 from PDF)
__global__ void solve_stream_function(double *Str, double *Vort, int phase) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if((i+j)%2 != phase) return;
    if(i <= 0 || i >= NX-1 || j <= 0 || j >= NY-1) return;

    Str[i*NY + j] = 0.25 * (
        Str[(i+1)*NY + j] + Str[(i-1)*NY + j] +
        Str[i*NY + (j+1)] + Str[i*NY + (j-1)] +
```

```
        DX*DX*Vort[i*NY + j]
    );
}

// Velocity Calculation (equations 34-35 from PDF)
__global__ void calculate_velocity(double *u, double *v, double *Str) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if(i <= 0 || i >= NX-1 || j <= 0 || j >= NY-1) return;

    const double inv2h = 1.0 / (2.0 * DX);

    // u = dψ/dy (equation 34)
    u[j*NX + i] = (Str[(j+1)*NX + i] - Str[(j-1)*NX + i]) * inv2h;

    // v = -dψ/dx (equation 35)
    v[j*NX + i] = -(Str[j*NX + (i+1)] - Str[j*NX + (i-1)]) * inv2h;

    // Apply top wall velocity
    if(j == NY-1 && i > 0 && i < NX-1) {
        u[j*NX + i] = U_Wall;
    }
}
```